

Lab Experiment 13

Minimax Algorithm for Gaming

Aim

To write a Python program to implement the Minimax algorithm for a two-player game.

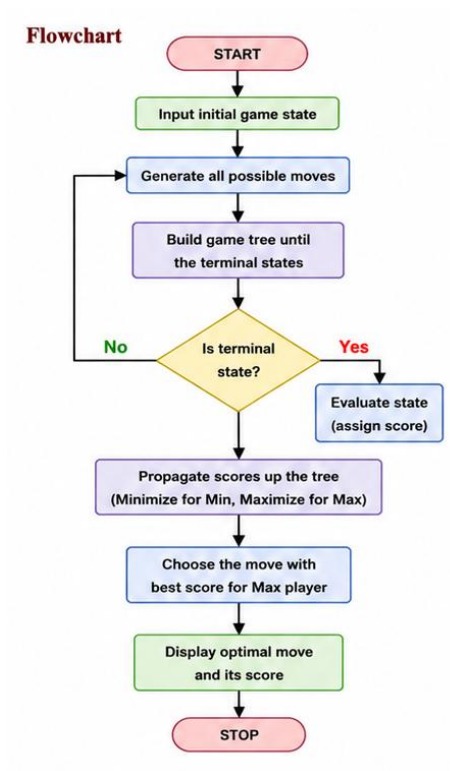
Objective

The objective of this experiment is to implement the Minimax algorithm for game playing. It aims to determine the optimal move by considering the possible moves of both players.

Algorithm

1. Define the initial game state.
2. Generate all possible moves.
3. Build the game tree up to the terminal states.
4. Assign scores to the terminal states.
5. Propagate the scores using the Minimax rule.
6. Maximize the score for the maximizing player and minimize it for the opponent.
7. Select and display the optimal move.

Flowchart



Code

Minimax Algorithm for Tic Tac Toe

```
def print_board(board):  
    for row in board:  
        print(" | ".join(row))  
    print("-" * 9)
```

```
def winner(board):  
    lines = []  
  
    # Rows  
    lines.extend(board)
```

```
# Columns
```

```
lines.extend([[board[r][c] for r in range(3)] for c in range(3)])
```

```
# Diagonals
```

```
lines.append([board[i][i] for i in range(3)])
```

```
lines.append([board[i][2-i] for i in range(3)])
```

```
for line in lines:
```

```
    if line[0] != ' ' and all(x == line[0] for x in line):
```

```
        return line[0]
```

```
if all(board[r][c] != ' ' for r in range(3) for c in range(3)):
```

```
    return 'Draw'
```

```
return None
```

```
def minimax(board, is_max):
```

```
    result = winner(board)
```

```
    if result == 'X':
```

```
        return 10
```

```
    elif result == 'O':
```

```
        return -10
```

```
    elif result == 'Draw':
```

```
        return 0
```

```
if is_max:
    best = float('-inf')

    for r in range(3):
        for c in range(3):
            if board[r][c] == ' ':
                board[r][c] = 'X'
                score = minimax(board, False)
                board[r][c] = ' '
                best = max(best, score)

    return best

else:
    best = float('inf')

    for r in range(3):
        for c in range(3):
            if board[r][c] == ' ':
                board[r][c] = 'O'
                score = minimax(board, True)
                board[r][c] = ' '
                best = min(best, score)

    return best
```

```
def best_move(board):
    best_value = float('-inf')
    move = (-1, -1)

    for r in range(3):
        for c in range(3):
            if board[r][c] == ' ':
                board[r][c] = 'X'
                move_value = minimax(board, False)
                board[r][c] = ' '

                if move_value > best_value:
                    best_value = move_value
                    move = (r, c)

    return move, best_value
```

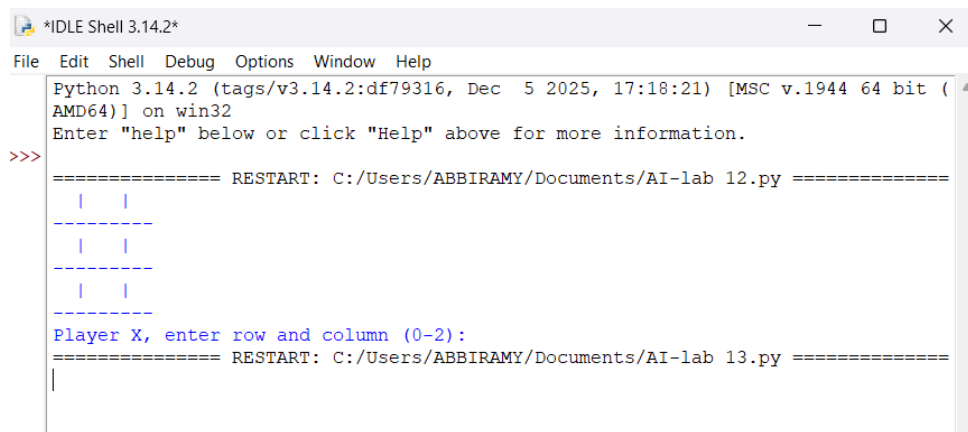
```
board = [[' ' for _ in range(3)] for _ in range(3)]
```

```
move, score = best_move(board)
```

```
print("Optimal Move for X:", move)
```

```
print("Minimax Score:", score)
```

Output



```
*IDLE Shell 3.14.2*
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ABBIRAMY/Documents/AI-lab 12.py =====
| | |
-----
| | |
-----
| | |
-----
Player X, enter row and column (0-2):
===== RESTART: C:/Users/ABBIRAMY/Documents/AI-lab 13.py =====
|
```

Result

The Python program successfully implemented the Minimax algorithm and determined the optimal move for the given game state.

Conclusion

The Minimax algorithm helps in decision-making for two-player games by considering possible future moves and selecting the best move assuming optimal play from both players.